Project : System Design Architecture:

1. Requirements Gathering:

In this phase we work on write down the Functional and Non-Functional Requirements for the project.

attachment:16e8faaf-42e2-43c7-8998-b8a7e5f68eca:Invoice_Management_System_-_FR_and_NFR_-_Mohammad_Diab_(1).pdf

2. Core Entities:

Here will build the database (schema - tables - ER Diagrams), also will go deep down into choosing ORM

ORM:

ORM is like a middleware between Backend Core Objects (OOP) and the Databases(usually SQL) that maps the objects fields to database tables and columns

PostegreSQL:

Database Server can be used in :

- Supabase
- neon
- Docker container on local machine
- AWS Instant

- Railway
- Prisma

Prisma and Prisma-client:

tools like CLI, Migrate, Driver, Adapter (PG) we do a configuration in prisma.config.ts + Schema.prisma

Railway:

1. Model definition: first `npx prisma generate` this will generate *generate/models/tableName.ts* second `npx prisma db push`
2. Migrations: like a version control for databases, use `npx prisma migrate dev –name migrateName`
3. Relations: connection between two models (primary key) handle one-to-one, one-to-many and many-to-many
4. Seedings: these are initial data that we can test with them, use `npx prisma db seed`

Data Modeling:

1. Indexing:

is related to the querying, and it is answer to some FR, only added after reviewing/monitoring system performance and latency responses

2. ER- Diagram:

use dbDiagram.io, this tool uses DSL language to define the database schema

after ER-Diagram design we will be moving to the API's

3. API:

every API should contact the database to get/post/delete/patch(update)

API should be divided according to its functionality

4. High Level Design:

must satisfy Functional Requirements, in this step we build the server (backend + API internal calls)

5. Deep Dives:

must satisfy Non Functional Requirements

Architecture:

